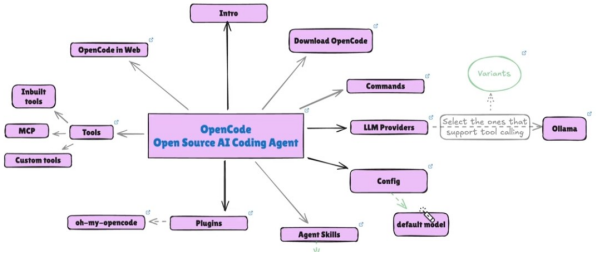
AI Tools

Yogesh Haribhau Kulkarni

OpenCode

OpenCode Ecosystem



(Ref: OpenCode CRASH Course - Data Science Basics)

What is OpenCode?

- Open-source coding agent built for the terminal (TUI).
- **IDE Integration possible**: Works with VS Code, Cursor, and any terminal-supporting IDE
- **GitHub Integration**: Automate issues and PRs directly from comments
- **Model-Agnostic:** Supports 75+ AI models (OpenAI, Claude, Groq, Ollama, and more).
  - *Local:* Ollama (fully offline, zero data egress)
  - *Cloud:* OpenAI, Claude, Groq, Gemini, etc.
- **Privacy-Focused:** Runs locally or uses your own API keys, your code stays on your machine.
- **Built With:** Rust/Go for performance; Bubble Tea for the TUI layer.

Capabilities

Think of it as having a senior developer sitting next to you

- Writing and debugging code
- Understanding existing codebases
- Refactoring and optimization
- Answering technical questions
- Automating repetitive tasks

Key Features

- **Interactive TUI:** Smooth terminal UI built with Bubble Tea; keyboard-driven workflow.
- **Contextual Understanding:** Reads your entire repo, understands dependencies, structure, languages.
- **Agentic Workflows:** Plans changes, searches files, executes shell commands, modifies code autonomously.
- **Plan / Build Modes/Agents:** Review proposed changes before applying, deterministic control.
- **Slash Commands:** `/init`, `/help`, `/models`, `/share`, and more.
- **Context Mentions:** `@file:`, `@workspace` for scoped AI queries.

Core Integration

- **LSP Integration:** Language Server Protocol for real-time code intelligence (completions, diagnostics, go-to-def).
- **MCP (Model Context Protocol):** Plug in external tools and data sources to extend agent capabilities.
- **ACP Support:** Agent Communication Protocol for inter-agent coordination.
- **Agent Skills & Custom Tools:** Define reusable agent behaviors and wire in custom shell/API tools.
- **Rules & Permissions:** Fine-grained control over what the agent can read, write, or execute.
- **Multi-Provider Routing:** Failover and cost-based routing across model providers.

OpenCode vs Other Tools

Feature	OpenCode	Claude Code	Cursor
Open Source	✅	❌	❌
Free Tier	✅	Limited	Limited
Terminal-first	✅	✅	❌
Provider Agnostic	✅	❌ (Claude only)	❌
Local Models	✅	❌	✅
IDE Integration	✅	✅	Native

(Ref: OpenCode - nxcode.io

Installation

- **Quick Install (Recommended):** `$ curl -fsSL https://opencode.ai/install | sh`
- **Install npm/bun:** `npm i -g opencode-ai@latest` or `bun add -g opencode-ai`
- **Desktop App:** Download directly from opencode.ai/download for a native desktop experience.

Getting Started

- **First Step:** Run `/init` to ground the agent in your project.
- **Configure Provider**: `Global config ~/.config/opencode/opencode.json Or project-specific ./opencode.json`
- **Config Format:** TOML-based `opencode.toml`; supports project-level and global overrides.
- **Bring Your Own Keys:** Set `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, or point to Ollama.
- **Launch:** `opencode` in any project directory. Have conda env activated before.

Plan vs Build Mode

Switch between them using the Tab key.
Plan Mode:

- Read-only - Cannot make changes to your code
- Analyzes and explores your codebase
- Suggests implementation strategies
- Perfect for understanding before modifying

Build Mode (Default)

- Full access - Can read, write, and modify files
- Executes code changes
- Runs commands
- Best for actual development work

Pro Tip: Always start complex features in Plan mode to outline your approach, then switch to Build mode for implementation.

Configuration

Overview

- OpenCode is fully configurable at **two scopes**: Global (`~/.config/opencode/`) and Project (`.opencode/` in repo root).
- **Config priority** (highest → lowest): CLI flags → Custom env (`OPENCODE_CONFIG_DIR`) → Project `opencode.json` → Global `opencode.json` → Remote org defaults (`.well-known/opencode`).
- **Subdirectories** (plural names preferred): `agents/`, `skills/`, `tools/`, `modes/`, `commands/`, `themes/`, `plugins/`.
- **Schema-validated:** Config is validated against `opencode.ai/config.json` — editors autocomplete fields automatically.
- **Claude Code compatibility:** Falls back to `CLAUDE.md` / `~/.claude/` conventions if OpenCode files are absent.
- Each section below covers: what ships built-in → what you can customize → how to create your own.

Customization Layers — The Mental Model

- **The core problem:** Context is a finite budget. Dumping everything into one file degrades quality — the agent buries critical constraints under noise. OpenCode solves this with *tiered, purposeful* context loading.
- **AGENTS.md — "The Onboarding Doc":** *Always-on*, repo-committed, team-shared. Tells *any* agent how this project works: stack, build commands, test steps, conventions. Think of it as the README for AI — written once, applied everywhere.
- **Rules — "Standing Orders":** *Always-on*, injected into every session. Global rules (`~/.config/opencode/AGENTS.md`) encode personal preferences never shared with the team. Multi-file rules via `opencode.json` allow conditional loading per glob/path. Same intent as AGENTS.md — just scoped differently (personal vs. project vs. package).
- **Skills — "The On-Call Expert":** *Lazy-loaded* — the agent reads only the name + description at startup, then fetches full content only when the task is relevant. Prevents token waste on instructions that don't apply to the current task. Think: nobody memorizes the entire wiki before starting a ticket.
- **Agents — "The Specialist Persona":** Own model, temperature, tool set, permissions, and system prompt. Invoked by `@mention` or as a subagent. Not just instructions — a *different actor* with a different role (e.g., a read-only `Review` agent vs. a write-enabled `Build` agent).
- **The layering principle:** Rules = what to always know. Skills = what to know when needed. Agents = who is doing the work and with what capabilities.

Customization Layers — When to Use What

- **Use AGENTS.md when:** The instruction is *project-wide*, stable, and useful to every agent, every session — build steps, test commands, folder conventions, PR rules. Commit it to Git; share with the team. Auto-generate with `/init`.
- **Use Rules when:** The instruction is always relevant but *not* project-specific — personal tone, preferred libraries, output format, privacy rules. Global rules stay on your machine, never in the repo.
- **Use Skills when:** The instruction is *task-specific* and only occasionally needed — e.g., a `git-release` skill for changelogs, a `db-migration` skill for schema changes. Keeps context lean; agent loads it only when it matches the task.
- **Use Custom Agents when:** You need a *different set of capabilities* for a role — a `Review` agent that can't write files, a `Debug` agent with only bash + read, a `Security` agent using a different model.
- **Use Commands (/) when:** You have a *repetitive prompt* — a parameterized template you type often. Commands invoke agents, pass arguments, and save keystrokes.
- **Summary table:**

Layer	Loaded	Scope
AGENTS.md	Always	Project / Package
Rules	Always	Personal / Team
Skills	On-demand	Any
Agents	When invoked	Session role
Commands	When typed	Shortcut

AGENTS.md

AGENTS.md helps OpenCode understand: project structure, Coding conventions, Technology stack etc

```
## Project: My App

## Tech Stack
- Next.js 14 with App Router
:

## Conventions
- Use functional components
:

## Structure
- /app - Next.js app router pages
- /components - Reusable UI components
:
```

Rules — Persistent AI Instructions

- **What:** Plain-text instructions injected into every LLM context. Think of them as standing orders — coding style, conventions, constraints.
- **Built-in bootstrap:** `/init` auto-generates `AGENTS.md` by scanning your repo — languages, deps, folder structure, patterns.
- **Scope:**
  - *Project rules*: `./AGENTS.md` — commit to Git; shared across team.
  - *Global rules*: `~/.config/opencode/AGENTS.md` — personal prefs, never committed.
  - *Claude Code fallbacks*: `CLAUDE.md` / `~/.claude/CLAUDE.md` if no `AGENTS.md` exists.
- **Multi-file rules** via `opencode.json`: glob patterns and remote URLs supported.
- **Lazy loading:** Reference `@rules/style.md` in `AGENTS.md` — agent reads files on-demand, not upfront (saves tokens).
- **Create custom:** Write any Markdown file → list it under `instructions` in `opencode.json`. No special syntax — plain instructions work.

```
{ "instructions": ["docs/standards.md",
    "test/guidelines.md", "packages/*/AGENTS.md",
    "https://myorg.com/team-rules.md"] }
```

Agents — Built-in & Custom

- **Two types:** *Primary agents* (you interact with directly, cycle via `Tab`) and *Subagents* (specialists invoked by primary agents or `@mention`).
- **Built-in Primary Agents:**
  - `Build` — default; all tools enabled; full file + shell access.
  - `Plan` — read-only; all mutating tools set to `ask`; safe for analysis.
- **Built-in Subagents:**
  - `General` — full tool access; multi-step parallel tasks.
  - `Explore` — read-only codebase navigator; fast file/pattern search.
- **Common custom agents:** `Review` (read-only + docs), `Debug` (bash + read), `Docs` (write, no shell), `Security` (audit mode).

- **Create custom:** Run `opencode agents create` → CLI prompts for scope, description, tools, model → writes `.opencode/agents/<name>.md`.
- **Agent frontmatter** fields: `model`, `temperature`, `mode` (`primary/subagent/all`), `tools`, `permission`, `steps` (max iterations), `color`.
- **Set default agent** in `opencode.json`: `"default_agent": "build"` — applies across TUI, CLI, IDE, GitHub Action.

Tools — Built-in & Custom

- **What:** Functions the LLM calls to act on your codebase. All built-in tools are enabled by default.
- **Built-in Tools:**
  - `bash` — execute shell commands (`git`, `npm`, `make`, etc.)
  - `edit` / `multiedit` — exact string replacement in files
  - `write` — create/overwrite files
  - `read` — read file contents (supports line ranges)
  - `grep` — regex search across codebase
  - `glob` — find files by pattern (`**/*.ts`)
  - `ls` — list directory contents
  - `patch` — apply unified diff patches
  - `webfetch` — fetch URLs during sessions
  - `skill` — load a `SKILL.md` on-demand
  - `todo` — manage task lists for multi-step work
  - `lsp` — code intelligence (defs, refs, hover) [experimental]
- **Custom Tools:** TypeScript/JS files in `.opencode/tools/` (project) or `~/.config/opencode/tools/` (global). Filename = tool name. Can wrap scripts in any language (Python, Bash, Go, etc.).
- **Override built-ins:** Custom tool with same name replaces the built-in (e.g., `bash.ts` → sandboxed shell).

```
// .opencode/tools/db-query.ts
import { tool } from "@opencode-ai/plugin"
export default tool({ description: "Run SQL query",
  args: { query: tool.schema.string() },
  async execute(args) { /* call your DB */ }
})
```

Agent Skills — Reusable Behavior Packages

- **What:** Markdown instruction bundles loaded on-demand by agents via the `skill` tool. Unlike rules (always loaded), skills are lazy — the agent picks them when relevant.
- **No built-in skills ship with OpenCode** — skills are community/team-created. Claude Code-compatible skills from `~/.claude/skills/` work as fallbacks.
- **Discovery locations** (all checked at startup):
  - `.opencode/skills/<name>/SKILL.md` — project-local
  - `~/.config/opencode/skills/<name>/SKILL.md` — global
  - `.claude/skills/` and `.agents/skills/` — compatibility paths
- **Name rules:** Lowercase alphanumeric + single hyphens; 1–64 chars; must match directory name.
- **Permissions:** Control skill access per agent with `allow` / `deny` / `ask` and wildcard patterns (`internal-*`).
- **Create a custom skill:**

```
# .opencode/skills/git-release/SKILL.md
---
name: git-release
description: Create consistent releases and
     changelogs
license: MIT
compatibility: opencode
---
## What I do
- Draft release notes from merged PRs
- Propose a version bump
- Generate a ‘gh release create‘ command
## When to use me
Use when preparing a tagged release.
```

Permissions — Granular Action Control

- **What:** Per-tool access policy — `allow` (auto-run), `ask` (prompt user), or `deny` (block). Defaults: most tools `allow`; `doom_loop` and `external_directory` default to `ask`; `.env` files denied by default.
- **Scope:** Set globally in `opencode.json`; override per-agent in agent frontmatter. Agent rules take precedence.
- **Runtime `always` approval:** During a session, approve a pattern once → auto-approved for the rest of the session.
- **External directory guard:** Use `external_directory` permission to allow/deny tool calls touching paths outside the working directory.
- **Skill permissions:** Controlled separately — grant/-deny specific skills by name or pattern.
- **Wildcard global policy + overrides** (last matching rule wins):

```
{ "permission": {
    "*": "ask",          // ask for everything by
     default
    "bash": {
      "*": "ask",
      "git *": "allow",  // git commands
     auto-approved
      "npm *": "allow",
      "rm *": "deny"     // destructive: always
     blocked
    },
    "edit": {
      "*": "deny",
      "src/**/*.ts": "allow"  // only allow edits
     in src/
    },
    "mymcp_*": "ask"   // all MCP tool calls:
     prompt first
  }
}
```

Modes — Contextual Tool Profiles

- **What:** Named configurations combining a system prompt + tool subset + temperature. Switch modes to change agent behavior without switching agents.
- **Difference from Agents:** Agents are full AI configurations with their own identity and model; Modes are lightweight overlays applied on top of an agent.
- **Built-in Modes:**
  - `Build` (default) — all tools on, full write access
  - `Plan` — analysis only; mutating tools set to `ask`
- **Useful custom modes:** `Review` (read-only + docs), `Debug` (bash + read, no writes), `Refactor` (edit only, no shell), `Docs` (write files, no system commands).
- **Create custom mode** — Markdown file in `.opencode/modes/` or `~/.config/opencode/modes/`:
- **Switch modes** in TUI: `Tab` key cycles between `Build` and `Plan` by default.

```
---
name: debug
description: Investigate issues, no file changes
tools:
  bash: true
  read: true
  grep: true
  write: false
  edit: false
temperature: 0.2
---
You are in debug mode. Diagnose and report only.
Do NOT modify any files. Use bash and read tools
to investigate. Output a structured findings
    report.
```

Commands — Slash-Invokable Shortcuts

- **What:** Reusable prompt templates invoked with `/` in the TUI. Eliminate repetitive prompt writing for common workflows.
- **Built-in commands:**
  - `/init` — scan project, generate `AGENTS.md`
  - `/models` — browse and switch active model
  - `/connect` — authenticate a new provider
  - `/share` — generate a shareable session URL
  - `/help` — list all available commands
- **Create custom commands** in `opencode.json` (or as `.opencode/commands/<name>.md` files):
- `$ARGUMENTS`: Anything typed after `/command-name` is injected here — makes commands parameterized.
- **Team sharing:** Commit `.opencode/commands/` to Git — entire team gets the same shortcuts.

```
{ "command": {
    "test-report": {
      "description": "Full test suite with
     coverage",
      "template": "Run the full test suite with
        coverage report and show any failures.
        Suggest fixes for failing tests."
    },
    "new-component": {
      "description": "Scaffold a React component",
      "template": "Create a new React component
        named $ARGUMENTS with TypeScript support.
        Include proper typing and basic structure."
    }
  }
}
```

MCP Servers — External Tool Integration

- **What:** Model Context Protocol (MCP) servers expose external tools and data sources to OpenCode. Any MCP-compatible server becomes a tool the agent can call natively.
- **Why:** Extends OpenCode beyond the filesystem — databases, APIs, cloud services, monitoring tools, internal systems.
- **Popular MCP servers:** GitHub, GitLab, Slack, PostgreSQL, Puppeteer, Brave Search, Filesystem (extended), AWS, Docker, Linear, Jira, Notion, Sentry.
- **Configure in `opencode.json`:**
- **Tool naming:** MCP tools are prefixed by server name: `github_create_issue`, `postgres_query`, etc. Use `mymcp_*` in permissions to control all tools from one server.
- **Org defaults:** Deploy company-wide MCP servers via `.well-known/opencode` — auto-loaded for all authenticated users.
- **Build your own:** Any HTTP/stdio process implementing the MCP spec becomes a server — SDKs available in TypeScript, Python, Go.

```
{ "mcp": {
    "github": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y",
     "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_TOKEN": "$GITHUB_TOKEN" }
    },
    "postgres": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y",
     "@modelcontextprotocol/server-postgres",
                "postgresql://localhost/mydb"]
    }
  }
}
```

Custom Tools — Extending the Agent's Hands

- **What:** TypeScript/JS files that define new functions the LLM can call. Can wrap any language internally (Python, Bash, Rust, etc.).
- **Placement:** `.opencode/tools/<name>.ts` (project) or `~/.config/opencode/tools/<name>.ts` (global).
- **One file = one or more tools:** Single default export → tool named after file. Multiple named exports → tools named `<file>_<export>`.
- **Multi-tool file example:**
- **Context available:** `context.agent`, `context.session`, `context.directory` (working dir), `context.worktree` (git root).
- **Override built-ins:** Same filename as a built-in (e.g., `bash.ts`) replaces the built-in completely — useful for sandboxing or auditing.
- **Any language via shell:** Call Python/Bash scripts inside `execute()` using `child_process` — tool definition is TS, implementation is anything.
- **Schema validation:** Use `tool.schema` (Zod) for type-safe arg definitions with `.describe()` for LLM-facing documentation.

```
// .opencode/tools/math.ts
import { tool } from "@opencode-ai/plugin"
export const add = tool({ description: "Add two
    numbers",
  args: { a: tool.schema.number(), b:
    tool.schema.number() },
  async execute({ a, b }) { return a + b }
})
export const multiply = tool({ description:
    "Multiply",
  args: { a: tool.schema.number(), b:
    tool.schema.number() },
  async execute({ a, b }) { return a * b }
})
// creates: math_add, math_multiply
```

Models — Provider & Model Configuration

- **75+ models supported** across: Anthropic, OpenAI, Google Gemini, Groq, Mistral, Cohere, xAI, DeepSeek, Qwen, Ollama (local), Hugging Face, AWS Bedrock, Azure, Vertex AI, and more.
- Chose models with long context window and have access to tools (function calling)
- **Switch models** in TUI: `/models` command → browse and select. Or `-m provider/model` CLI flag at launch.
- **Configure in `opencode.json`:**
- **Variants:** Define multiple configurations of the same model (different temperature, max_tokens, reasoning effort) and cycle with `variant_cycle` keybind.
- **Per-agent model:** Each custom agent can specify its own model — e.g., `Plan` uses a cheap fast model; `Build` uses full Sonnet/Opus.

- **Small model for meta-tasks:** OpenCode uses a lightweight model (default: `gpt-5-nano` via Zen) for session titling and other housekeeping — configurable separately.
- **Authenticate:** `/connect` in TUI → select provider → paste API key. Supports OAuth for GitHub/GitLab providers.

```
{ "model": "anthropic/claude-sonnet-4-6",
  "provider": {
    "anthropic": { "apiKey": "$ANTHROPIC_API_KEY"
    },
    "openai":    { "apiKey": "$OPENAI_API_KEY" },
    "ollama":    { "url": "http://localhost:11434"
    }
  }
}
```

Real-World Use Cases

- **Understanding a Codebase:** `You: Explain the authentication flow in this project` OpenCode will analyze your files and provide a detailed explanation of how authentication works.
- **Adding a Feature:** `You: Add a dark mode toggle to the settings page` OpenCode will: Find the settings page, Identify the theme system, Implement the toggle, Update necessary styles
- **Debugging:** `You: The login form is not submitting. Here's the error: [paste error]` OpenCode will trace the issue and suggest fixes.
- **Multi-File Refactoring:** Rename, restructure, or migrate patterns across the entire codebase atomically.
- **Documentation Generation:** Auto-generate README, API docs, and inline comments from code context.
- **Test Generation:** Scaffold unit/integration tests with awareness of existing test patterns.
- **Security Auditing:** Identify unsanitized inputs, SQL injection points, or insecure defaults.

Demo

Initialize & Understand

- **Goal:** Show how OpenCode scans and understands a new project from scratch.
- **Setup:** Open terminal inside any Git repository.
- **Launch:** `$ opencode`
- **Command:** `> /init`
- **What Happens:** OpenCode scans all local files, infers stack (languages, deps, structure), and writes `AGENTS.md`, the AI's instruction manual for this project.
- **Key Point:** Every subsequent interaction is grounded in this context, no hallucinated file paths.

Fix & Refactor

- **Goal:** Demonstrate AI-driven code modification on a real file.
- **Command:** `> @file:main.go fix the bug where user input isn't sanitized`
- **Agent Action:** Reads `main.go`, identifies the unsanitized input vector, proposes a targeted patch.
- **Review Flow:**
  - Press `Tab` → switch to **Plan** mode: inspect the proposed diff before anything touches disk.
  - Press `Tab` → switch to **Build** mode: apply the change.
- **Key Point:** You stay in control, the agent proposes, you decide.

Tips for Beginners

- Be Specific: Don't say "Make it better" but say, "Refactor the handleSubmit function to use async/await and add error handling"
- Provide Context: Don't say "Fix the bug" but say, "The login form throws 'undefined is not a function' when clicking submit. The error occurs in auth.ts line 45"
- Use Plan Mode First: For complex features, ask OpenCode to create a plan before implementation.
- Leverage Image Support: Drag and drop screenshots or mockups into the terminal—OpenCode can understand images!
- Create Project-Specific Configs: Different projects may need different settings. Use local opencode.json files.

Summary

Why Use OpenCode?

- **Privacy-First:** Code never leaves your machine by default; no telemetry, no vendor data retention.
- **Cost Efficiency:** No subscription fees, pay only for actual API token usage; local models are free.
- **No Vendor Lock-in:** Switch AI providers in seconds via config; one tool, all models.
- **Terminal-Native Speed:** Faster context-switch than IDE plugins; integrates directly into your existing shell workflow.
- **Transparent & Auditable:** Fully open-source (Apache 2.0), inspect, fork, or extend anything.
- **Enterprise-Ready:** Supports self-hosted LLMs and air-gapped (physically isolated from all other networks) environments.
- **Bottom Line:** Faster dev loop, zero lock-in, full code privacy, AI that works *with* your terminal.

Quick Reference — Config Cheat Sheet

- **File locations** (P = project `.opencode/`, G = global `~/.config/opencode/`):
  - Rules: `P: AGENTS.md` · `G: AGENTS.md`
  - Agents: `P: agents/<name>.md` · `G: agents/<name>.md`
  - Skills: `P: skills/<name>/SKILL.md` · `G: skills/<name>/SKILL.md`
  - Tools: `P: tools/<name>.ts` · `G: tools/<name>.ts`
  - Modes: `P: modes/<name>.md` · `G: modes/<name>.md`
  - Commands: `opencode.json → "command": {...}`
  - MCP: `opencode.json → "mcp": {...}`
  - Permissions: `opencode.json → "permission": {...}`
- **Key CLI commands:**
  - `opencode agents create` — scaffold a new agent interactively
  - `/init` — generate `AGENTS.md` for current project
  - `/models` — browse and switch models
  - `/connect` — authenticate a new provider
  - `/share` — share current session URL
- **Priority order** (highest → lowest): CLI flags → `OPENCODE_CONFIG_DIR` → `.opencode/opencode.json` → `~/.config/opencode/opencode.json` → `.well-known/opencode` (org defaults).
- **Docs:** opencode.ai/docs · **GitHub:** github.com/anomalyco/opencode

Resources

- GitHub: `github.com/opencode-ai/opencode`
- Docs: `opencode.ai/docs`
- NXCode.io "OpenCode Tutorial 2026: Install & Setup After Anthropic Block (Updated Guide)"

OpenWork by diffeernt-ai

What is OpenWork?

- **One-Line Definition:** An open-source, local-first desktop AI agent framework — a guided-workflow GUI and control surface for knowledge workers, built on top of OpenCode.
- **Origin Story:** Created by different-ai as a flexible, model-agnostic alternative to proprietary agentic tools like Claude Code, designed to put users in control of their own AI workflows.
- **What It Does:** Instead of responding one prompt at a time, OpenWork *plans, executes, and delivers* finished work — automating file manipulation, task orchestration, and multi-step agent workflows.
- **Built on OpenCode:** OpenWork acts as a thin, user-friendly GUI layer on top of OpenCode (opencode.ai), with UI actions mapping directly to OpenCode API calls.
- **Status:** Open-source (MIT License) · Desktop app powered by Tauri (Rust + Web) · Available on GitHub at `github.com/different-ai/openwork`.
- **Market Signal:** OpenWork launched in January 2026, weeks after Claude Code, and rapidly drew developer attention as the leading open-source agentic desktop alternative.

OpenWork vs. Claude Cowork vs. Accomplish AI

- **Claude Cowork (Anthropic):** Proprietary · Anthropic models only · Cloud-centric · No source access. Targets non-developers via a polished GUI.
- **OpenWork — different-ai:** Open-source (MIT) · Model-agnostic · Local-first · Direct OpenCode SDK integration. Targets developers and technical users.
- **Accomplish AI (formerly accomplish-ai/openwork):** Open-source · Spawns OpenCode CLI as a subprocess via Electron · Aimed at non-technical users · Rebranded to Accomplish.ai.
- **Key Differentiator:** OpenWork by different-ai integrates directly with the OpenCode SDK (not a subprocess), offering the highest architectural fidelity to OpenCode's design philosophy.
- **Ejectable:** Power users can drop from the GUI to the underlying OpenCode CLI at any time — no lock-in.
- **Tech Stack:** Tauri (Rust + web frontend) — lightweight, native, and cross-platform; versus Electron (Node.js) used by Accomplish AI.

Core Philosophy & Design Principles

- **Local-First, Cloud-Ready:** OpenWork runs on your machine. Workflows execute locally by default; cloud connectivity is opt-in, not mandatory.
- **Model Agnostic:** Bring your own API keys. Switch freely between local models, OpenAI, DeepSeek, or any OpenCode-compatible provider — no ecosystem lock-in.
- **Parity with OpenCode:** Every UI action maps directly to an OpenCode API call. There are no hidden intermediary layers or proprietary abstractions.
- **Transparency:** Audit Logs capture agent activity. File tracking shows exactly what the agent read, wrote, or modified during a task.
- **User Approval for Dangerous Actions:** Plans, permissions, and progress are exposed to the user. Potentially destructive operations require explicit approval.
- **Ejectable Design:** If the GUI is insufficient for your use case, switch to OpenCode CLI — your configuration and workflow definitions transfer seamlessly.

Getting Started — Installation & Setup

- **Prerequisites:** Node.js and pnpm (for building); OpenCode installed from `opencode.ai`.
- **Step 1 — Install OpenCode:** Configure your LLM provider(s) and API keys within OpenCode. OpenWork inherits all provider settings from OpenCode automatically.
- **Step 2 — Launch OpenWork:** Open the desktop app. Select your operating mode:
  - *Host Mode:* Runs OpenCode locally and acts as the server for agent execution.
  - *Client Mode:* Connects to a trusted, running host via URL, token, or QR code — useful for remote or shared setups.
- **Step 3 — Configure Providers:** Set up local models (e.g., Ollama), OpenAI, DeepSeek, or any supported provider. Switch at any time without reinstalling.
- **Development Workflow:** Use `pnpm dev` to start the full dev stack. Ensure all submodules are updated with `git submodule update --init --recursive`.
- **Building:** `pnpm build` compiles the Tauri app for your platform.

How OpenWork Executes Tasks

- **Step 1 — Task Intake:** Type your goal into the chat interface. OpenWork passes it to OpenCode, which analyzes the request and generates a structured execution plan.
- **Step 2 — Plan Review & Approval:** The plan is surfaced in the UI. You review each proposed step before execution begins — no silent automation.
- **Step 3 — Execution:** OpenCode executes the plan using its tool surface (file read/write, shell commands, browser automation, external integrations).
- **Step 4 — File Handling:** Attach files directly or provide local file paths. The agent can read, edit, summarize, and generate project documents autonomously.
- **Step 5 — Deliver & Track:** Outputs land in your specified location. The Audit Log records every action taken, every file touched, and every decision made.
- **Commands:** Frequently used plans can be saved as reusable *Commands* — re-run parameterized workflows without re-prompting from scratch.

Key Capabilities

- **Local File Access:** Read, write, create, and organize files on your machine — no upload/download cycle, no cloud dependency.
- **Model Flexibility:** Switch between local models (Ollama), OpenAI GPT-4o, DeepSeek Reasoner, or any OpenCode-compatible provider per task.
- **Skills Marketplace:** Extend functionality with prebuilt skills — browser automation (Chrome MCP), scheduling, external tool integration, and more.
- **DeepSeek Integration:** For complex tasks requiring multi-step reasoning, OpenWork can delegate to DeepSeek Reasoner — including pivoting from browser automation to local script generation mid-task.
- **Audit Log:** Full, tamper-evident record of agent actions. Know exactly what ran, what changed, and when.
- **Reusable Commands:** Package common workflows as saved commands for instant re-execution with new parameters.

- **OpenWork Apps (Tauri SDK):** A framework within the app for developers to build native applications using OpenCode and Tauri — extending OpenWork as a platform, not just a tool.
- **60fps UI:** Architecture enforces a non-blocking main thread; heavy computations run off-thread to keep the interface responsive.

Audit Log & Transparency

- **What the Audit Log Captures:**
  - Every file read, written, or deleted during agent execution
  - Every shell command or external API call made
  - Timestamps and model decisions at each step
  - Plan approvals and user interventions
- **Why It Matters:** Unlike proprietary agentic tools, OpenWork exposes the full execution trace. There are no black-box operations.
- **File Tracking:** The UI surfaces a live file-change feed during task execution. See what the agent is doing in real time.
- **Dangerous Action Gate:** Actions that could cause data loss (e.g., file deletion, irreversible writes) require explicit user `Allow` confirmation before proceeding.
- **Compliance Advantage:** For teams that cannot send data to proprietary cloud agents, OpenWork's local-first + audit trail design provides a practical path to deploying AI automation within existing data governance policies.
- **Open Source Guarantee:** Because the codebase is MIT-licensed, any organization can audit the agent's behavior at the source level — not just through logs.

Skills Marketplace & Extensibility

- **What Skills Are:** Pre-built capability packages that extend OpenWork's default tool surface. Install a skill to add a new category of agent actions.
- **Available Skill Categories:**
  - *Browser Automation* — Chrome MCP integration for web scraping, form filling, and UI-level automation
  - *Scheduling* — trigger workflows on a time-based schedule
  - *External Tool Integration* — connect to APIs, webhooks, and third-party services
  - *Script Generation* — fallback from browser automation to local scripts for robust task completion
- **OpenWork Apps:** The Tauri SDK within OpenWork lets developers build *native applications* on top of OpenCode + Tauri — treating OpenWork as a platform.
- **Plugin Install Flow:**
  1. Open OpenWork → navigate to `Skills` panel
  2. Browse or search the marketplace
  3. Click `Install` → skill is immediately available in the agent's tool surface
- **Community-Driven:** Skills are community-contributed and open-source — inspect, fork, or publish your own.

Model Agnosticism — Bring Your Own Keys

- **Supported Providers (via OpenCode):**
  - *Local:* Ollama (run any open-weight model fully on-device — no API key needed)
  - *OpenAI:* GPT-4o, GPT-4o-mini, and compatible models
  - *DeepSeek:* DeepSeek Chat and DeepSeek Reasoner for complex reasoning tasks
  - *Any OpenCode-compatible provider* — configuration is inherited directly from OpenCode
- **Per-Task Model Selection:** Choose the most appropriate (or cost-effective) model for each task without reconfiguring the app.
- **Cost Control:** Use local models (zero API cost) for routine file operations; switch to frontier models only for tasks that require it.
- **No Vendor Lock-In:** If a provider's pricing changes or a better model is released, switch without migrating your workflows or data.
- **DeepSeek Reasoner Use Case:** For complex tasks requiring multi-hop reasoning or dynamic strategy changes (e.g., "if browser automation fails, write a local script instead"), DeepSeek Reasoner is the recommended model within OpenWork.

OpenWork vs. Proprietary Agents — Feature Comparison

Feature	OpenWork (different-ai)	Claude
Open Source	Yes (MIT)	N
Model Support	Agnostic (Local / API)	Anthrop
Primary Use	Desktop Agent / Workflow	Codebase & Kn
Execution	Local-First	Cloud-C
Audit Log	Yes (full trace)	N
Ejectable to CLI	Yes (OpenCode CLI)	N
Pricing	Free (BYOK)	Paid pla
Tech Stack	Tauri (Rust / Web)	Propri
Skills / Plugins	Community Marketplace	11 Anthrop

Demo 1 — First Launch & Setup

- **Goal:** Show onboarding flow and provider configuration.
- **Step 1:** Install OpenCode from `opencode.ai`; add your API key(s) to OpenCode config.
- **Step 2:** Clone and build OpenWork:

```
git clone
    https://github.com/different-ai/openwork
cd openwork
git submodule update --init --recursive
pnpm install && pnpm build
```

- **Step 3:** Launch OpenWork → select `Host` mode → confirm OpenCode server is running.
- **Step 4:** Navigate to the provider selector — switch from OpenAI to a local Ollama model to demonstrate model agnosticism.
- **What to Show:** The agent tool surface loading, the Audit Log panel initializing, and the Skills Marketplace listing available extensions.
- **Key Message:** Full local setup, zero vendor dependency, and a live audit trail — from clone to running agent in minutes.

Demo 2 — File Automation Task

- **Goal:** Show local file access, plan approval, and transparent execution.
- **Setup:** Place a folder of mixed documents (PDFs, CSVs, markdown notes) in a local directory.
- **Task Prompt:**

```
> Read all files in ~/openwork-demo.
  Summarize each document in one paragraph,
  extract any numerical data into a CSV,
  and produce a master index file with links
  to each summary.
```

- **What Happens:** OpenWork generates a structured plan. User reviews and approves. Agent reads files, writes summaries, generates CSV, creates index — Audit Log captures every step.
- **Key Message:** The plan is visible before execution, every file touch is logged, and no data ever leaves the local machine.

Demo 3 — Reusable Command Workflow

- **Goal:** Show the Commands feature for repeatable, parameterized workflows.
- **Setup:** A recurring task — weekly competitive intelligence report from a list of URLs.
- **First Run — Define the Workflow:**

```
> Fetch the pages in urls.txt, extract key
  product announcements and pricing changes
  from the past 7 days, and write a
     structured
  Markdown report saved to reports/.
```

- **Save as Command:** After the first successful run, save the plan as the command `/weekly-intel`.
- **Subsequent Runs:** Type `/weekly-intel` → approve the pre-built plan → done. No re-prompting required.
- **Key Message:** OpenWork learns your recurring workflows. Commands turn one-time tasks into repeatable, auditable automation — without writing any code.

Demo 4 — Browser Automation via Skills

- **Goal:** Show the Skills Marketplace extending OpenWork beyond local file operations.
- **Setup:** Install the *Browser Automation (Chrome MCP)* skill from the marketplace.
- **Task Prompt:**

```
> Go to the pricing pages for the top 5 AI
  coding tools listed in competitors.txt.
  Extract plan names, prices, and feature
  limits. Save results to pricing_table.csv.
```

- **What Happens:** OpenWork uses the Chrome MCP skill to launch a controlled browser session, navigates each page, extracts structured data, and writes the CSV — switching to local script generation if a page blocks automation.
- **What to Show:** The Audit Log capturing each browser action, the skill badge appearing in the task trace, and the final CSV delivered locally.
- **Key Message:** Skills turn OpenWork into a general-purpose automation platform without sacrificing auditability or local-first execution.

Summary: Why OpenWork Matters

- **Paradigm Shift:** From "AI that answers questions" to "AI that executes workflows" — open, auditable, and local-first.
- **Open Source by Design:** MIT license. No vendor lock-in. Inspect, fork, and deploy on your own terms. Community-driven skills and plugins.
- **Model Freedom:** Run entirely on local models (zero API cost), switch to frontier models when needed — all with the same UI and workflow definitions.
- **Transparent Execution:** Every agent action is logged, every plan is shown before execution, and every dangerous operation requires explicit approval.
- **Built for Extensibility:** The Skills Marketplace and OpenWork Apps SDK let developers build on top of OpenWork as a platform, not just use it as a tool.
- **Closest to OpenCode's Philosophy:** Direct SDK integration (not a subprocess wrapper) means OpenWork reflects OpenCode's architecture faithfully — a GUI that enhances rather than abstracts.
- **The Bigger Picture:** OpenWork is the open-source answer to "What does AI automation look like when *you* own the stack?"

Resources & Links

- **GitHub Repository:** github.com/different-ai/openwork
- **OpenCode (Agent Engine):** opencode.ai
- **License:** MIT — github.com/different-ai/openwork/blob/main/LICENSE
- **Issues & Contributions:** github.com/different-ai/openwork/issues
- **different-ai Organization:** github.com/different-ai
- **Skills Marketplace:** Available within the OpenWork desktop app under the `Skills` panel.
- **Build from Source:**

```
git clone
    https://github.com/different-ai/openwork
git submodule update --init --recursive
pnpm install && pnpm dev   # development
pnpm build                 # production
```

Google NotebookLM

What is NotebookLM?

- **One-Line Definition:** A source-grounded AI research assistant by Google Labs — it answers questions, synthesizes insights, and generates multimodal outputs *exclusively from documents you provide*.
- **Origin Story:** First introduced in May 2023 as *Project Tailwind* at Google I/O. Rebranded as NotebookLM in 2024 and rolled out broadly to researchers, students, and enterprise users.
- **The Core Idea:** Unlike general chatbots that draw on broad training data, NotebookLM is grounded in *your* sources — minimizing hallucinations and providing inline citations linking directly to source passages.
- **Underlying Model:** Powered by Google Gemini (upgraded to Gemini 2.0 Flash in Dec 2024; Gemini 3 in 2025) — with higher model tiers available for paid plans.
- **Availability:** Web app at notebooklm.google.com + mobile app · 180+ regions · 50+ languages · Free to use with a Google account.
- **Scale:** Globally, millions of people and tens of thousands of organizations use NotebookLM; users have generated more than 350 years' worth of Audio Overviews.

The Three-Panel Interface

- **Redesigned Dec 2024:** NotebookLM introduced a three-column layout that organizes the entire research workflow into three focused areas.
- **Sources Panel (Left):** The central knowledge base for your project. Add, manage, and browse all uploaded sources. Click any source to open the full document viewer. Select specific sources to scope chat responses.
- **Chat Panel (Center):** Conversational AI interface grounded in your sources. Ask questions, request summaries, or generate custom content. Every response includes clickable inline citations that jump directly to the relevant passage in the original document.
- **Studio Panel (Right):** One-click content generation from your sources — Audio Overviews, Video Overviews, Mind Maps, Reports, Flashcards, Quizzes, Slide Decks, Infographics, and Study Guides. All outputs are saved in the Studio for ongoing access.
- **Adaptive Layout:** Expand the source viewer and notes editor side by side for deep reading, or collapse panels to focus on chat while an Audio Overview plays in the background.

Supported Source Types

- **Documents:** PDFs (up to 200MB / 500,000 words each), Google Docs, Google Slides, Google Sheets, Microsoft Word (`.docx`), plain text files. Note: copy-protected PDFs will fail to import.
- **Web & Video:** Website URLs (public pages; paywalled or scraping-blocked sites will fail), YouTube video links (NotebookLM extracts the transcript automatically).
- **Audio:** MP3, WAV, AAC, OGG, OPUS, and 20+ other formats containing clear speech (music-only files are not supported).
- **Images:** Upload images as sources for visual content analysis.
- **Notes:** Manually authored notes within a notebook can be converted into sources, letting you build on AI analysis with your own insights.
- **Google Drive Integration:** Pull in Google Docs, Slides, and Sheets directly from Drive — no manual download required.
- **Limitations:** Excel files are not directly supported (export as PDF as a workaround). Sources are static — updated versions must be manually replaced.

Audio Overviews — The Viral Feature

- **What It Is:** Upload any set of documents and NotebookLM generates a podcast-style conversation between two lifelike AI hosts who summarize, debate, and contextualize your content.
- **Launched Sep 2024:** Immediately viral — Spotify used it for personalized 2024 Wrapped podcast overviews. Users generated over 350 years of Audio Overviews in the first three months.
- **Interactive Mode (Dec 2024):** "Join the conversation" — tap the Join button to ask the AI hosts questions mid-overview and receive personalized, real-time responses.
- **Customization (2025):** Choose from four formats — *Deep Dive*, *Brief*, *Critique*, or *Debate*. Adjust tone (formal/humorous) and length. Inject custom instructions to steer the hosts' focus.
- **80+ Languages (Aug 2025):** Expanded from English-only to 80+ languages with full depth parity. Audio Overviews now match the quality of the original English version across all supported languages.
- **Scope Control:** Generate an Audio Overview from all sources in your notebook, or select a specific subset for a focused summary.

Video Overviews, Slide Decks & Infographics

- **Video Overviews (2025):** AI hosts discuss your source material while presenting relevant AI-generated visuals, diagrams, charts, and quotes — creating a narrated slide-style video. Available in six visual styles powered by Google's Nano Banana image generation model and in formats: *Brief* and *Explainer*. Available in 80+ languages.
- **Slide Decks (Nov 2025):** Generate a structured presentation directly from your sources — complete with layouts, visuals, and narrative text. Long-form deck option available on higher tiers. Watermarks appear on Free/Plus/Pro tiers; removed for Ultra subscribers.
- **Infographics (Nov 2025):** Create visually rich summary graphics from your uploaded sources. Powered by Nano Banana image generation. Watermarks removed for Ultra tier.
- **Export:** Slide Decks and Infographics can be exported and shared. Mind Maps export as PNG (for presentations) or JSON (for integrations).
- **Multiple Outputs:** The Studio Panel stores multiple generated outputs of the same type within a single notebook — iterate freely without overwriting previous versions.

Mind Maps, Reports & Learning Tools

- **Mind Maps (Mar 2025):** One click generates an interactive visual map of key concepts and connections extracted from your sources. Nodes expand multi-level deep; click any node to open a targeted AI chat on that topic. Export as PNG or JSON.
- **Reports:** Ask NotebookLM to synthesize findings on a specific question across all sources — delivering a cited, structured briefing document. Useful for literature reviews, executive briefings, and competitive analysis.
- **Data Tables (Dec 2025):** Synthesize scattered data across sources into clean, structured tables — exportable directly to Google Sheets. Use cases: clinical trial comparisons, action items from meeting transcripts, competitor pricing matrices, exam study tables.
- **Flashcards:** Auto-generated from source content; ideal for exam preparation and employee training.
- **Quizzes:** NotebookLM generates multiple-choice and open-ended assessments grounded in your uploaded materials.
- **Study Guides & Timelines:** Pre-built outputs for academic workflows — chronological event timelines and structured study guides with key terms, questions, and summaries.

Deep Research — Autonomous Source Discovery

- **What It Is:** NotebookLM's agentic research mode. Give it a topic and it autonomously creates a research plan, searches hundreds of high-quality web sources, and compiles a comprehensive, citation-backed report — then adds those sources to your notebook.
- **Fast Research vs. Deep Research:** *Fast Research* surfaces ∼10 relevant documents in under 30 seconds. *Deep Research* runs a broader, multi-step crawl — slower but significantly more comprehensive.
- **Source Discovery Panel:** A dedicated panel within NotebookLM lets you search for sources from the web or from your Google Drive before deciding which to add.
- **Workflow Integration:** Combine external tools — e.g., use Gemini for broad web research, then feed the output into NotebookLM as a grounded source for focused Q&A and synthesis.
- **Plan Limits:** Free tier allows 10 Deep Research sessions per month; Pro allows up to 20 per day; Ultra allows up to 200 per day.
- **Important Caveat:** If information is not in your sources (uploaded or discovered), NotebookLM will not speculate. It will say so explicitly — a feature, not a bug.

Pricing & Plans

Feature	Free	Plus	Pro
Price	$0	Workspace	$19.99/mo
Notebooks	100	200	500
Sources / Notebook	50	100	300
Chat queries / day	50	2×	5×
Audio Overviews / day	3	2×	5×
Video Overviews / day	Limited	2×	5×
Deep Research / month	10/mo	2×	20/day
Slide Deck watermark	Yes	Yes	Yes
Model access	Gemini	Higher	Higher

- **Pro** is bundled with Google AI Pro (formerly Google One AI Premium) — also includes Gemini Advanced, AI in Gmail/Docs, and 2TB storage. US students get 50% off ($9.99/mo, first year).
- **Plus** is included with qualifying Google Workspace plans (starting at $14/user/month).
- **Ultra** also removes watermarks on Slide Decks and Infographics and grants priority feature access.

Enterprise & Collaboration Features

- **NotebookLM Enterprise (GA 2025):** Runs within the customer's own Google Cloud project. Data is not shared externally; processed inside VPC Service Controls and Identity Access Management (IAM). Suitable for regulated industries (legal, healthcare, finance).
- **Enterprise API (GA):** Automate notebook creation and management — e.g., generate a template notebook per new project, auto-insert RFPs, produce initial report skeletons programmatically.
- **Google Agentspace Integration:** Enterprises can deploy NotebookLM as an internal AI agent within Google Agentspace — treating it as a knowledge infrastructure layer, not just a single-user tool.
- **Collaboration:** Share notebooks with teammates as viewers or editors (similar to Google Docs sharing). Publish notebooks publicly. Notebooks can be scoped to only expose the chat/search interface to collaborators, keeping raw sources private.
- **Data Privacy Guarantee:** Uploaded materials, queries, and model responses are **not** used to train Google's models — on Free, Pro, and Workspace tiers. Confirmed by Google's product documentation.
- **Admin Controls:** Workspace admins can enable/disable NotebookLM at the domain, OU, or group level. Context-Aware Access (CAA) is supported.

Key Use Cases

- **Academic Research:** Accelerate literature reviews — upload papers, generate mind maps of themes, query across all PDFs simultaneously with cited answers, produce timelines and study guides. Every answer links back to the exact passage.
- **Enterprise Knowledge Management:** Upload compliance manuals, product docs, HR policies. New hires query the notebook instead of hunting through documents. Onboarding time reduced significantly.
- **Sales & Presales:** Digest RFPs, specs, and past proposals; generate tailored meeting plans; produce audio/video briefings to prepare for customer conversations.
- **Executive Intelligence:** Upload financial reports, market analyses, and strategy documents; generate executive summaries, extract metrics, and identify strategic implications.
- **Content Production:** Turn research bundles into audio guides, infographics, slide decks, and video explainers — without manual formatting work.
- **IT & Engineering:** Upload production logs and post-mortem documents; generate root cause timelines and mind maps of system dependencies; query issues across hundreds of pages of logs instantly.
- **Education:** Generate quizzes, flashcards, and study guides from course materials; create audio overviews for on-the-go learning.

Limitations & Honest Caveats

- **Cloud-Only:** No offline mode on any plan. All processing requires an active internet connection and happens on Google's infrastructure.
- **Single AI Provider:** Only Google Gemini models are available. You cannot bring your own API keys, switch to OpenAI or Claude, or run local models.
- **Static Sources:** Documents do not update automatically. When a source file changes, you must manually remove the old version and re-upload the new one.
- **No Chat History:** NotebookLM does not retain conversation history between sessions. Save important exchanges as notes and convert them to sources if needed.
- **Source-Bound Answers:** If information is not in your uploaded sources, NotebookLM will not answer from general knowledge. This is a deliberate design choice for accuracy, but limits open-ended queries.
- **Quality Degrades Near Source Limit:** Answer accuracy declines as you approach your plan's source cap — the model retrieves from an increasingly large pool, reducing precision.
- **Safety Filters:** Queries involving violence, sexuality, or obscenity in sources (even in historical contexts) may trigger safety flags and prevent responses.
- **Google Ecosystem Lock-In:** Deep integration with Drive, Docs, Sheets, and Slides is a strength for Google Workspace users but a constraint for Microsoft 365-centric organizations.

Demo 1 — Research Synthesis in Minutes

- **Goal:** Show core source grounding, cited Q&A, and Audio Overview generation.
- **Setup:** Create a new notebook. Add 5–10 PDFs on a research topic (e.g., AI agent architectures) plus two or three YouTube video URLs and a Google Doc of notes.
- **Step 1 — Explore:** Click "Generate" on the Sources panel. NotebookLM suggests relevant questions based on the uploaded material — select one.
- **Step 2 — Q&A with Citations:** Ask:

```
> What are the key differences in memory
  architecture between the papers?
```

  Show inline citations — click one to jump to the exact passage in the source PDF.
- **Step 3 — Audio Overview:** Studio panel → Audio Overview → choose "Deep Dive" format. Play the generated podcast. Demonstrate the "Join" button to ask a live question to the AI hosts mid-overview.
- **Key Message:** From raw documents to a listenable, cited research briefing in under two minutes — with zero hallucination risk.

Demo 2 — Mind Map & Data Table

- **Goal:** Show visual synthesis and structured data extraction.
- **Setup:** Notebook with 10–15 market research reports and competitor analysis PDFs.
- **Step 1 — Mind Map:** Studio panel → Mind Map. Show the auto-generated concept graph. Click a top-level node (e.g., "Pricing Strategies") to expand sub-nodes. Click a sub-node to open a targeted chat: the AI answers questions scoped to that concept cluster.
- **Step 2 — Data Table:** In the Chat panel, type:

```
> Create a table comparing each competitor's
  pricing tier, feature set, and target
  customer segment from the sources.
```

  Show the structured table output. Click "Export to Google Sheets" to transfer it directly.
- **What to Highlight:** The mind map and table are generated from the same source set — zero manual data entry, full citation traceability.
- **Key Message:** NotebookLM turns a folder of reports into an interactive knowledge graph and a structured data asset simultaneously.

Demo 3 — Slide Deck & Video Overview

- **Goal:** Show multimodal output generation for presentations and async communication.
- **Setup:** Notebook with a Deep Research report, three supporting PDFs, and a meeting transcript.
- **Step 1 — Slide Deck:** Studio panel → Slide Deck → choose "Long" format (Pro/Ultra). Show the generated presentation with layouts, AI-generated visuals, and source-derived text. Note the watermark (removed on Ultra).
- **Step 2 — Video Overview:** Studio panel → Video Overview → choose visual style (e.g., "Explainer"). Play the AI-narrated video with slides, diagrams, and quotes drawn from sources.

- **Step 3 — Output Language:** Demonstrate switching the output language selector to Japanese — regenerate the briefing document in Japanese from the same English sources.
- **What to Show:** All three outputs (Slide Deck, Video, Translated Briefing) generated from a single source set in under five minutes.
- **Key Message:** NotebookLM is not just a research tool — it is a multimodal content production engine grounded in verified sources.

Demo 4 — Enterprise: Onboarding & Deep Research

- **Goal:** Show collaborative notebooks and autonomous source discovery for enterprise workflows.
- **Setup:** A shared team notebook with product specs, onboarding manuals, and policy documents (Google Workspace account — data stays within org trust boundary).
- **Step 1 — Collaborative Q&A:** Share the notebook with a colleague as a viewer (chat-only access; sources remain private). Show the colleague asking a policy question and getting a cited answer without seeing the underlying documents.
- **Step 2 — Deep Research:** In a separate research notebook, use the Source Discovery panel → Deep Research. Enter:

```
> Competitive landscape for enterprise AI
  research assistants, Q4 2025
```

  Show NotebookLM building a bibliography autonomously, then generate a cited briefing report.
- **Step 3 — Admin Controls:** Briefly show the Google Workspace Admin console toggle for enabling/disabling NotebookLM at the domain level.
- **Key Message:** Enterprise-grade access control, data governance, and collaborative research — all within the Google Workspace security boundary.

NotebookLM vs. Competing Tools

Feature	NotebookLM	ChatGPT Projects	Claude
Source Grounding	Yes (RAG, cited)	Partial	Partial
Inline Citations	Yes (passage-level)	No	No
Audio Podcast Output	Yes	No	No
Video Overview	Yes	No	No
Mind Map	Yes	No	No
Slide Deck Generation	Yes	No	No
Deep Research Agent	Yes	Yes (web)	Yes (web)
Model Choice	Gemini only	GPT only	Claude only
Offline / Local	No	No	No
Free Tier	Yes	Limited	Yes
Data Not Used for Training	Yes	Opt-out	Yes

Summary: Why NotebookLM Matters

- **Paradigm Shift:** From "AI that knows everything" to "AI that knows *your* documents" — source-grounded, hallucination-resistant, and fully cited.
- **Multimodal Knowledge Engine:** One notebook, many outputs — audio podcasts, narrated videos, slide decks, infographics, mind maps, data tables, quizzes, and reports — all from the same source set.
- **Accessible & Free:** Core features available to anyone with a Google account. Powerful enough for serious research; simple enough for first-time AI users.
- **Scales to Enterprise:** VPC-SC, IAM, admin controls, and an enterprise API make it deployable in regulated environments where data cannot leave the organization.
- **Viral by Design:** Audio Overviews became a cultural moment — 350+ years of listening generated in the first months. Spotify integration. NotebookLM proved AI can be genuinely delightful.
- **Trajectory:** 2023–24 solved the input problem (RAG). 2025 solved the format problem (multimodal outputs). The roadmap points toward persistent memory, agentic orchestration (Google Agentspace), and ambient AI through Project Astra / XR devices.
- **The Bigger Picture:** NotebookLM is Google's answer to "What if AI only ever spoke from documents you trust?"

Resources & Links

- **Product:** `notebooklm.google.com`
- **Plans & Pricing:** `notebooklm.google/plans`
- **Help Center — FAQ:** `support.google.com/notebooklm → FAQ`
- **Help Center — Upgrade:** `support.google.com/notebooklm → Upgrade`
- **Google Workspace:** `workspace.google.com/products/note`
- **Workspace Updates Blog:** `workspaceupdates.googleblog.` (search: NotebookLM)
- **Google Labs Blog:** `blog.google → Google Labs` (latest release notes)
- **Mobile App:** Available on iOS and Android — core features including source upload via one-tap share, Audio Overview playback, and chat.

Copyleft ⓒ Send suggestions to yogeshkulkarni@yahoo.com